

UNTREF

UNIVERSIDAD NACIONAL
DE TRES DE FEBRERO

TRABAJO PRÁCTICO INTEGRADOR

Módulo 3

Docente:

Diego Wolf.

Alumno:

Nicolás Pannunzio.

Fecha: 1-06-2026

ÍNDICE

INTRODUCCIÓN.....	1
1. DESCRIPCION.....	1
2. OBJETIVOS	1
CONFIGURACIONES INICIALES.....	2
ENTORNO VIRTUAL –VENV–	2
DESARROLLO DE ACTIVIDADES.....	4
ACTIVIDAD I	4
ACTIVIDAD II.....	6
SELENIUM & PYTEST – ENTORNO DE PRUEBAS	6
AUTOMATIZACIÓN DE CASOS DE PRUEBA (WEB).....	7
TEST CASE 1.....	8
TEST CASE 2.....	10
TEST CASE 3.....	14
ACTIVIDAD III	17
CYPRESS – ENTORNO DE PRUEBAS	17
TEST CASE 1.....	19
TEST CASE 2.....	20
TEST CASE 3.....	21
CONCLUSIONES	22
HERRAMIENTAS UTILIZADAS	23
BIBLIOGRAFÍA Y REFERENCIAS.....	24

INTRODUCCIÓN

1. DESCRIPCION

El presente trabajo integrador corresponde al Módulo 3 de la Diplomatura en Control de Calidad de Software que dicta la Universidad Nacional de Tres de Febrero. Su finalidad es aplicar los conocimientos adquiridos sobre lógica de programación y diseño de arquitecturas de pruebas automatizadas. A lo largo del presente se desarrollarán diferentes algoritmos lógicos y suites de pruebas integrales (End-to-End y de API) sobre entornos reales como SauceDemo y PokeAPI. Esto permitirá demostrar el dominio de conceptos fundamentales como la manipulación dinámica del DOM con Selenium WebDriver, la ejecución y validación de peticiones HTTP mediante Cypress, y la gestión de aserciones y reportes de ejecución utilizando el framework Pytest.

2. OBJETIVOS

Los objetivos principales de este trabajo integrador son:

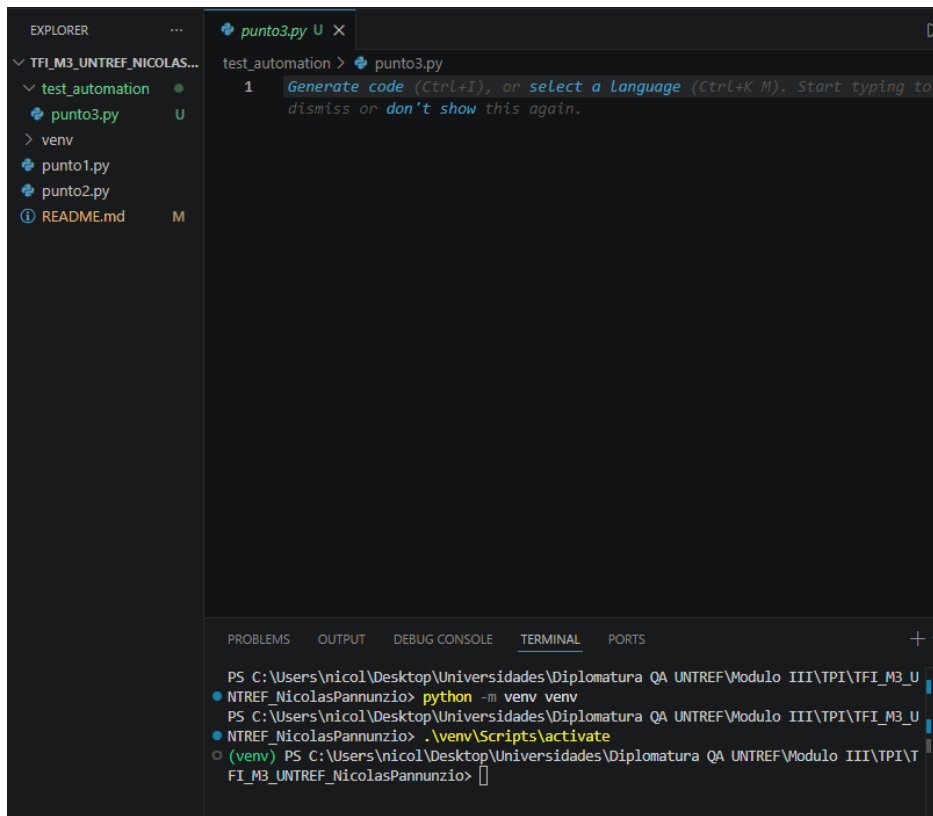
- Comprender y aplicar correctamente la lógica de programación en Python para la resolución de algoritmos matemáticos y el manejo de excepciones.
- Diseñar e implementar una suite de pruebas End-to-End (E2E) utilizando Python, Pytest y Selenium WebDriver para la validación de flujos de usuario en interfaces web dinámicas.
- Configurar y operar una arquitectura de trabajo híbrida, aislando y gestionando dependencias para entornos de Python (venv) y Node.js (npm).
- Automatizar peticiones HTTP (GET, POST) y validar respuestas estructurales JSON contra APIs REST reales utilizando el framework Cypress.

- Gestionar y aislar el estado de las aplicaciones bajo prueba (ej. limpieza de Local Storage y caché) para garantizar pruebas determinísticas.
 - Interpretar, documentar y reportar los resultados obtenidos mediante la generación automática de informes de ejecución HTML.
 - Documentar de manera clara y profesional la arquitectura del proyecto, sus instrucciones de ejecución y la resolución de cada consigna en el repositorio correspondiente.
-

CONFIGURACIONES INICIALES

ENTORNO VIRTUAL –VENV–

Para aislar el proyecto dentro de un entorno seguro, el primer paso es crear una carpeta dentro del proyecto y luego a través de los comandos que se ven en la consola, creo dicho entorno de trabajo.

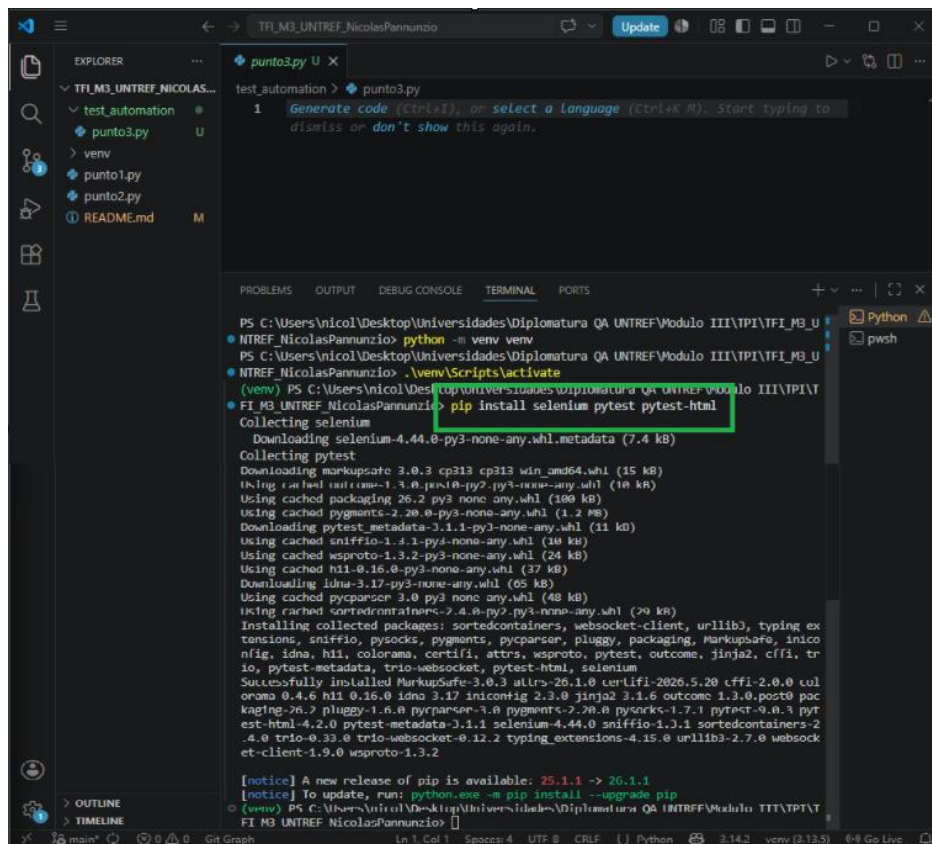


```
EXPLORER
...
TFI_M3_UNTREF_NICOLAS...
  test_automation
    punto3.py
  venv
    punto1.py
    punto2.py
    README.md

punto3.py
1 Generate code (Ctrl+I), or select a Language (Ctrl+K M). Start typing to dismiss or don't show this again.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TFI_M3_U
NTREF_NicolasPannunzio> python -m venv venv
PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TFI_M3_U
NTREF_NicolasPannunzio> .\venv\Scripts\activate
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\T
FI_M3_UNTREF_NicolasPannunzio>
```

Seguidamente, se instalan los paquetes de Python principales a través de un solo comando separados por un espacio y ejecutados a la vez.



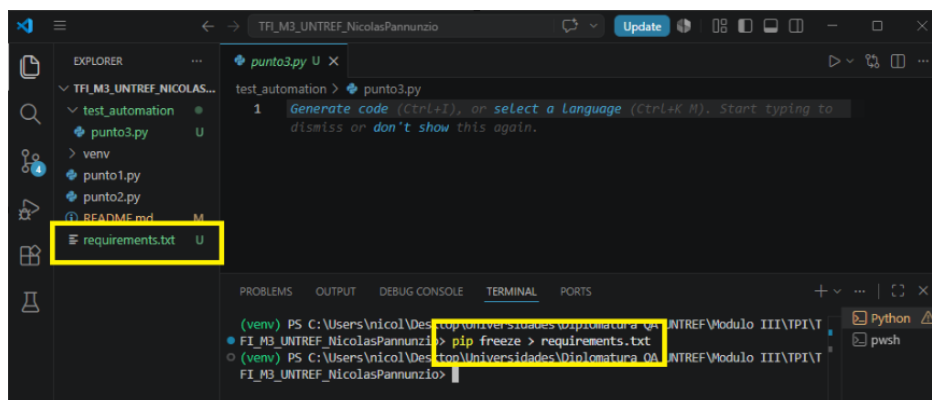
The screenshot shows a VS Code window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with files like `punto1.py`, `punto2.py`, `punto3.py`, and `README.md`. The terminal shows the following commands and output:

```
PS C:\Users\Nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\T...> python -m venv venv
PS C:\Users\Nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\T...> .venv\Scripts\activate
(venv) PS C:\Users\Nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\T...> pip install selenium pytest pytest-html
```

The output of the `pip install` command shows the installation of various packages, including `selenium`, `pytest`, and `pytest-html`.

El último paso antes de comenzar con la resolución de los algoritmos en Python, es crear el gestor de dependencias (`requirements.txt`). Su finalidad, es documentar las versiones exactas de las herramientas instaladas con el fin de que un equipo de trabajo o en este caso, el docente evaluador, al clonar el repositorio pueda reconstruir el entorno de pruebas idéntico ejecutando un solo comando:

pip install -r requirements.txt



The screenshot shows a VS Code window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with files like `punto1.py`, `punto2.py`, `punto3.py`, and `requirements.txt`. The terminal shows the following commands and output:

```
(venv) PS C:\Users\Nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\T...> pip freeze > requirements.txt
```

The output of the `pip freeze > requirements.txt` command shows the creation of the `requirements.txt` file.

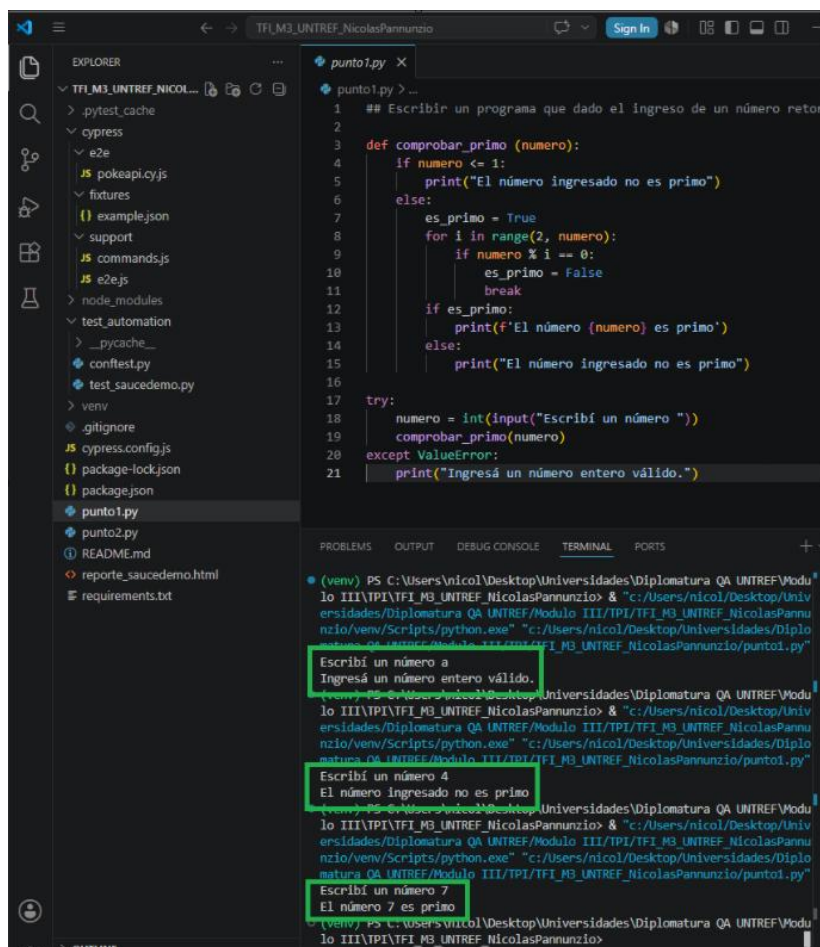
DESARROLLO DE ACTIVIDADES

ACTIVIDAD I

ALGORITMOS Y LÓGICA EN PYTHON

A. VALIDACIÓN DE NÚMEROS PRIMOS

Para el desarrollo del script punto1.py, el objetivo principal fue la creación de un algoritmo capaz de evaluar si número entero es primo (divisible por sí mismo y por uno) o no. Para conseguirlo integré un control de flujo para la interacción del usuario y un manejo de excepciones (bloques try-except) para interceptar entradas inválidas (como caracteres de texto, símbolos o valores nulos), garantizando que el programa devuelva mensajes de error controlados en lugar de colapsar la ejecución.



```
1  ## Escribir un programa que dado el ingreso de un número retorne si es primo o no
2
3  def comprobar_primo (numero):
4      if numero <= 1:
5          print("El número ingresado no es primo")
6      else:
7          es_primo = True
8          for i in range(2, numero):
9              if numero % i == 0:
10                 es_primo = False
11                 break
12             if es_primo:
13                 print(f'El número {numero} es primo')
14             else:
15                 print("El número ingresado no es primo")
16
17  try:
18      numero = int(input("Escribi un número "))
19      comprobar_primo(numero)
20  except ValueError:
21      print("Ingresá un número entero válido.")
```

Terminal output:

```
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio> & "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/venv/Scripts/python.exe" "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/punto1.py"
Escribi un número a
Ingresá un número entero válido.
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio> & "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/venv/Scripts/python.exe" "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/punto1.py"
Escribi un número 4
El número ingresado no es primo
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio> & "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/venv/Scripts/python.exe" "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/punto1.py"
Escribi un número 7
El número 7 es primo
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio>
```

B. RESOLUCIÓN DE ECUACIONES CUADRÁTICAS

El script punto2.py se estructuró bajo el paradigma de modularización, dividiendo el cálculo de la fórmula de Bhaskara (fórmula resolvente) en funciones independientes y reutilizables. A nivel de pruebas lógicas, el diseño se enfocó en el manejo de "casos límite" (Edge Cases). Se programaron validaciones matemáticas preventivas para evitar fallos críticos, tales como la división por cero (cuando el coeficiente 'a' equivale a 0) y el manejo condicional del discriminante negativo, informando al usuario la imposibilidad de calcular raíces reales sin interrumpir abruptamente el sistema.

```
1  ## Escribir una función que, dado el ingreso de 3 variables (a, b, c)
2  ## retorne las raíces resultantes de una ecuación cuadrática.
3
4  def discriminante(a, b, c):
5      resultado_disc = b ** 2 - 4 * a * c
6      if a == 0:
7          return "El primer valor ingresado no puede ser cero"
8      if resultado_disc > 0:
9          raiz_cuadrada = resultado_disc ** 0.5
10         primer_resultado = (- b + raiz_cuadrada) / (2 * a)
11         segundo_resultado = (- b - raiz_cuadrada) / (2 * a)
12         return f'La ecuación cuadrática tiene dos resultados posibles (pr
13     elif resultado_disc == 0:
14         tercer_resultado = (-b) / (2 * a)
15         return f'El resultado de la función cuadrática es {tercer_resulta
16     else:
17         return "La ecuación no tiene solución para los numeros ingresados"
18
19
20 a = int(input("Ingresá el primer número "))
21 b = int(input("Ingresá el segundo número "))
22 c = int(input("Ingresá el tercer número "))
23 print(discriminante(a,b,c))
```

```
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modu
lo III\TPI\TFI_M3_UNTREF_NicolasPannunzio> ^C
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modu
lo III\TPI\TFI_M3_UNTREF_NicolasPannunzio> & "c:/Users/nicol/Desktop/Univ
ersidades/Diplomatura QA UNTREF/Modulo III/TPI/TFI_M3_UNTREF_NicolasPannu
nzio/venv/Scripts/python.exe" "c:/Users/nicol/Desktop/Universidades/Diplo
matura QA UNTREF/Modulo III/TPI/TFI_M3_UNTREF_NicolasPannunzio/punto2.py"

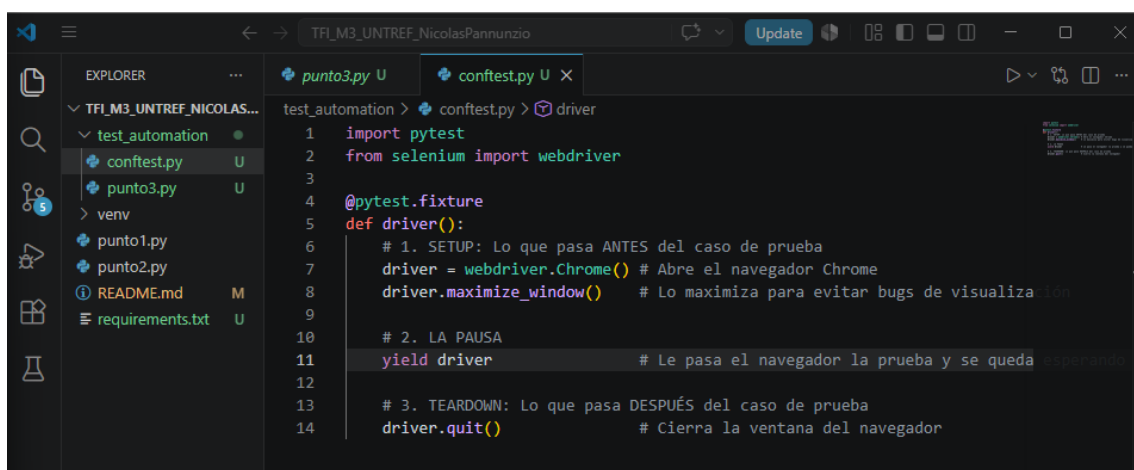
Ingresá el primer número 0
Ingresá el segundo número 5
Ingresá el tercer número 7
El primer valor ingresado no puede ser cero
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modu
lo III\TPI\TFI_M3_UNTREF_NicolasPannunzio>
```

ACTIVIDAD II

SELENIUM & PYTEST – ENTORNO DE PRUEBAS

Opté por implementar una configuración global a través de un archivo nativo – `conftest.py`– para centralizar la gestión del WebDriver. Además, mediante el uso de `@pytest.fixture` y la instrucción `yield`, conseguí aislar la lógica del setup (iniciar y maximizar el navegador) y el teardown (cierre de sesión).

Estas configuraciones además de ahorrar algunas líneas de código, garantizan que cada caso de prueba cuente con un entorno limpio en cada ejecución.



```
test_automation > conftest.py > driver
1  import pytest
2  from selenium import webdriver
3
4  @pytest.fixture
5  def driver():
6      # 1. SETUP: Lo que pasa ANTES del caso de prueba
7      driver = webdriver.Chrome() # Abre el navegador Chrome
8      driver.maximize_window()    # Lo maximiza para evitar bugs de visualización
9
10     # 2. LA PAUSA
11     yield driver                # Le pasa el navegador la prueba y se queda esperando
12
13     # 3. TEARDOWN: Lo que pasa DESPUÉS del caso de prueba
14     driver.quit()              # Cierra la ventana del navegador
```


AUTOMATIZACIÓN DE CASOS DE PRUEBA (WEB)

La propuesta en este trabajo integrador para realizar la automatización de los primeros test cases fue para el sitio web: <https://www.saucedemo.com>

Inspección de Elementos (DOM) y Estrategia de Localizadores:

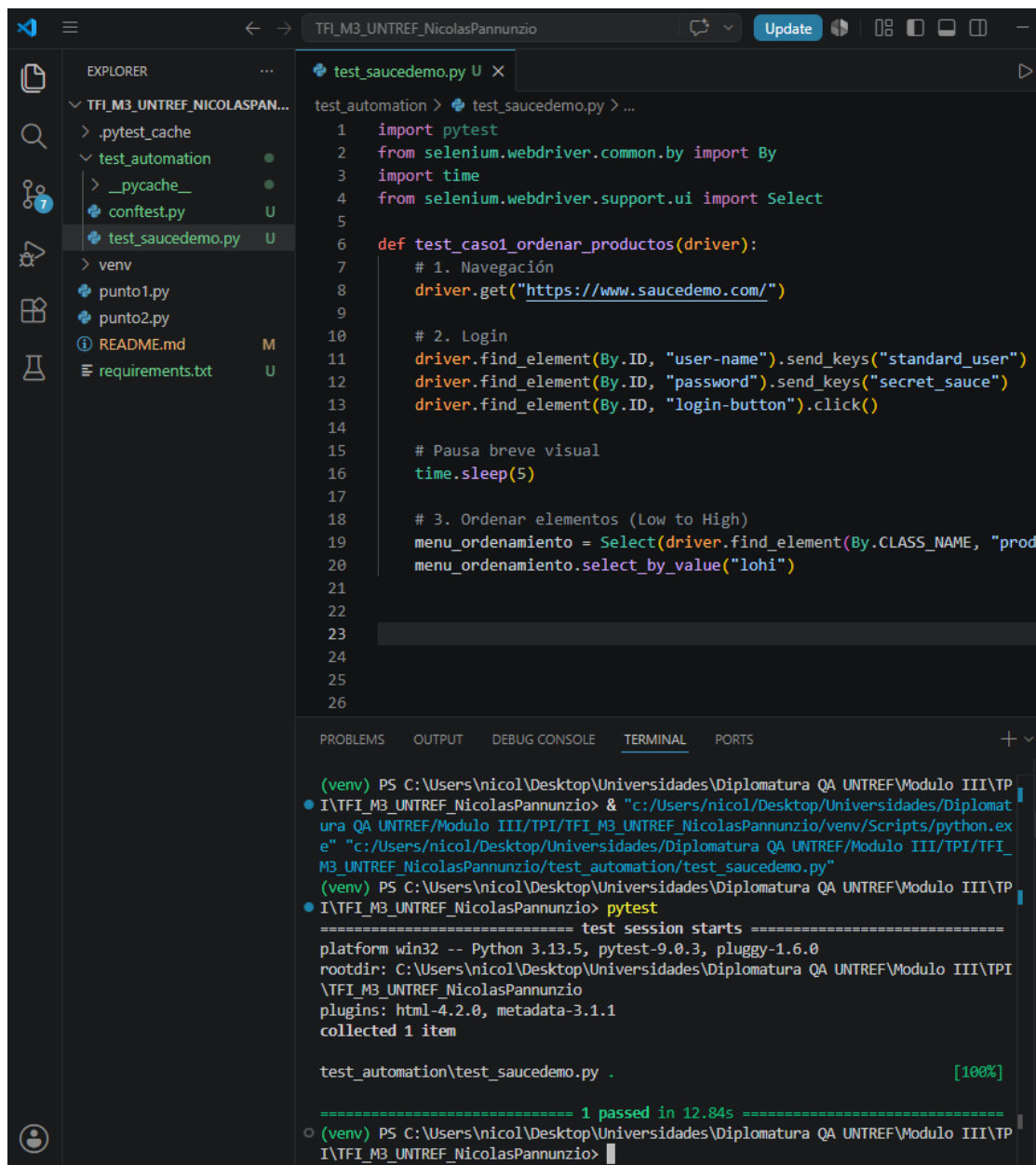
Para lograr que el script interactuara correctamente con la interfaz gráfica, fue fundamental analizar cómo estaba construida la página "por detrás". Para esto, recurrí constantemente a la consola de Herramientas para Desarrolladores del navegador (accesible mediante la tecla F12), lo que permitió inspeccionar el DOM en tiempo real y entender la estructura de la página.

A la hora de definir la estrategia de localizadores (la forma en que le indicamos a Selenium dónde hacer clic o qué campo llenar), prioricé inicialmente la búsqueda mediante el localizador By.ID, lo que garantiza que el WebDriver identifique el objetivo, reduciendo la posibilidad de pruebas frágiles o falsos positivos (flaky tests).

Sin embargo, asumiendo la realidad del desarrollo web donde no todos los elementos cuentan con un identificador único, la estrategia debió ser flexible y adaptarse a las posibilidades de cada caso de prueba. Cuando el atributo ID no estaba disponible o no era funcional para el objetivo (por ejemplo, al tener que interactuar con múltiples botones idénticos en el catálogo), se optó por la búsqueda mediante clases genéricas (By.CLASS_NAME). Esta combinación permitió sortear los obstáculos de la interfaz y lograr una cobertura exitosa en todos los escenarios planteados.

TEST CASE 1

Con esta premisa, ejecuté la primera fase del script automatizado mediante el framework Pytest. La consola evidencia un resultado exitoso (1 passed), confirmando que el WebDriver logró instanciar el navegador, resolver el inicio de sesión con las credenciales provistas y manipular exitosamente el elemento <select> para ordenar el catálogo.



The screenshot displays the Visual Studio Code interface. The Explorer panel on the left shows the project structure with files like `test_saucedemo.py` and `requirements.txt`. The main editor shows the content of `test_saucedemo.py`, which includes imports for `pytest`, `By`, `time`, and `Select`, and a test function `test_caso1_ordenar_productos`. The function performs three steps: navigation to the Saucedemo website, login with 'standard_user' and 'secret_sauce', a 5-second pause, and selecting 'lohi' from a dropdown menu.

```
1 import pytest
2 from selenium.webdriver.common.by import By
3 import time
4 from selenium.webdriver.support.ui import Select
5
6 def test_caso1_ordenar_productos(driver):
7     # 1. Navegación
8     driver.get("https://www.saucedemo.com/")
9
10    # 2. Login
11    driver.find_element(By.ID, "user-name").send_keys("standard_user")
12    driver.find_element(By.ID, "password").send_keys("secret_sauce")
13    driver.find_element(By.ID, "login-button").click()
14
15    # Pausa breve visual
16    time.sleep(5)
17
18    # 3. Ordenar elementos (Low to High)
19    menu_ordenamiento = Select(driver.find_element(By.CLASS_NAME, "product_sort_selector"))
20    menu_ordenamiento.select_by_value("lohi")
21
22
23
24
25
26
```

The TERMINAL panel at the bottom shows the command prompt output, indicating a successful test run:

```
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio> & "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/venv/Scripts/python.exe" "c:/Users/nicol/Desktop/Universidades/Diplomatura QA UNTREF/Modulo III/TPI/TPI_M3_UNTREF_NicolasPannunzio/test_automation/test_saucedemo.py"
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio> pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

test_automation\test_saucedemo.py . [100%]

===== 1 passed in 12.84s =====
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TPI_M3_UNTREF_NicolasPannunzio>
```

Mapeo de datos estructurados para aserciones

Para cumplir con el criterio de aceptación -"Verificar que los elementos estén ordenados"- se procedió a inspeccionar el DOM en busca de los valores numéricos.

Se identificó que todos los precios del catálogo comparten la clase `inventory_item_price`. Esta clase será utilizada para recolectar dinámicamente los textos web, transformarlos en datos numéricos (flotantes) y aplicar la validación lógica correspondiente.

The screenshot shows the Sauce Labs website with two products displayed: the Sauce Labs Backpack and the Sauce Labs Bike Light. The DOM inspector on the right side of the browser shows the HTML structure of the page. The class `div.inventory_item_price` is highlighted, which is used to identify the price of each item.

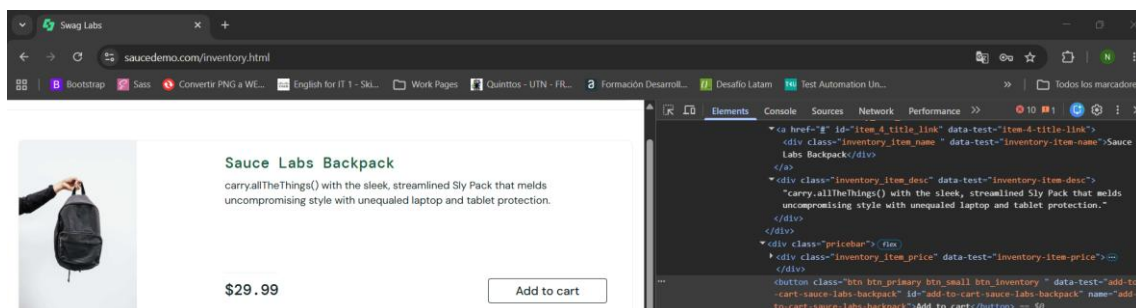
The screenshot shows a grid of six products on the Sauce Labs website. Each product card includes an image, a title, a description, and a price. The prices are displayed in a box with the class `div.inventory_item_price`. The products and their prices are:

- Sauce Labs Onesie: \$7.99
- Sauce Labs Bolt T-Shirt: \$15.99
- Sauce Labs Backpack: \$29.99
- Sauce Labs Bike Light: \$9.99
- Test.allTheThings() T-Shirt (Red): \$15.99
- Sauce Labs Fleece Jacket: \$49.99

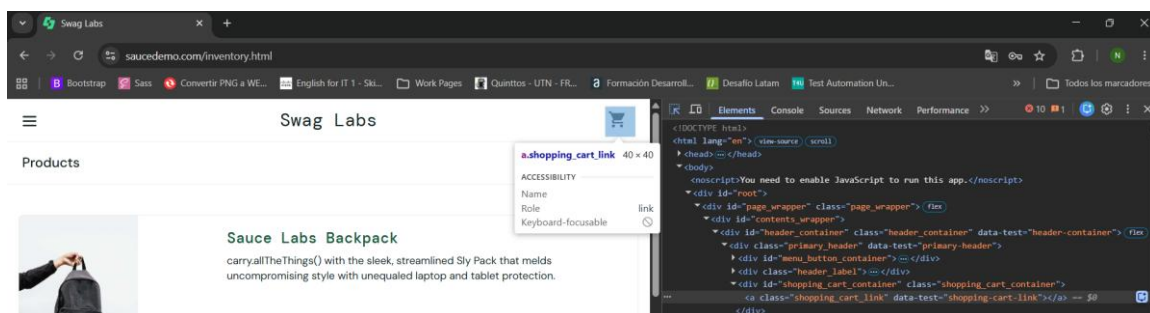
TEST CASE 2

Identificación de botones de compra (elementos de acción)

Se inspeccionó el código fuente para localizar los controles de adición al carrito. Si bien se observa que cada botón posee un atributo ID y name único asociado al producto específico, se identificó que todos los botones de compra comparten la clase genérica `btn_inventory`. Este patrón de diseño es fundamental, ya que permite utilizar dicha clase para capturar la colección completa de elementos web y ejecutar acciones en bloque.

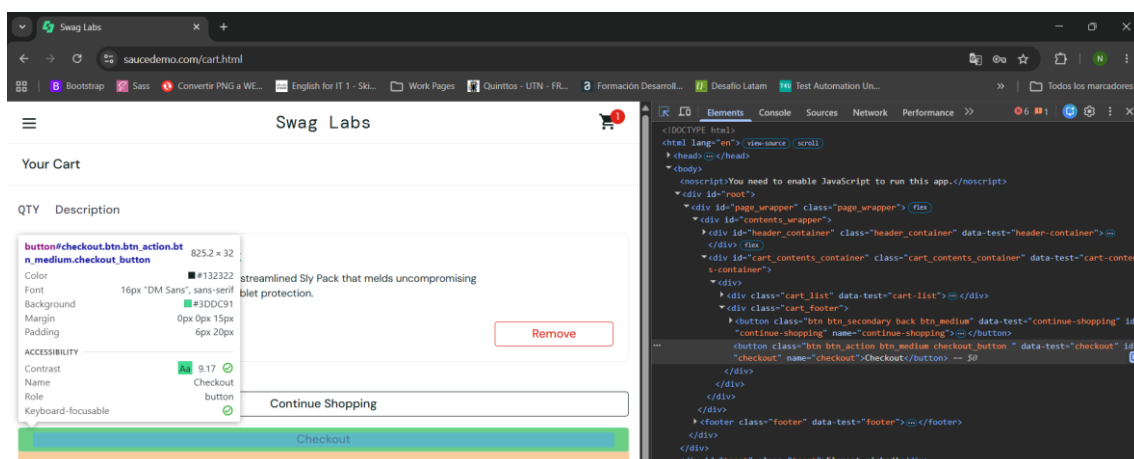
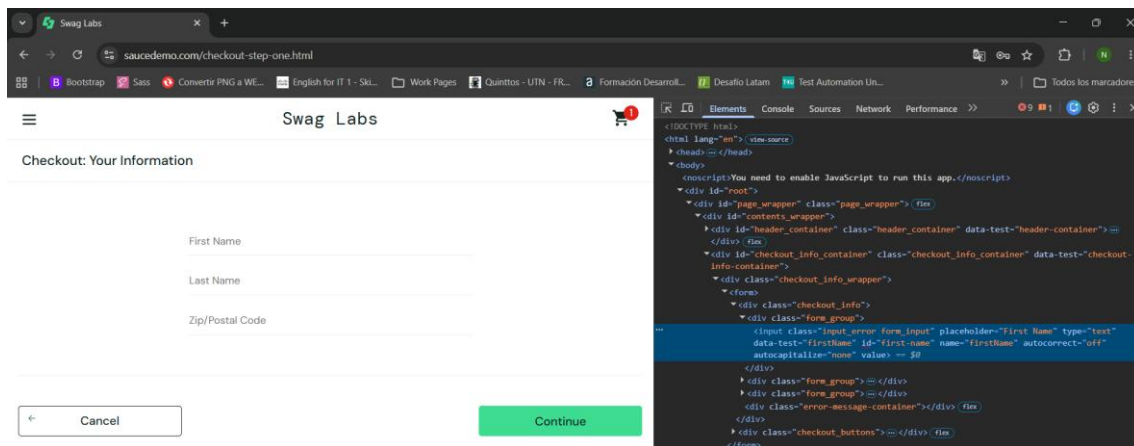


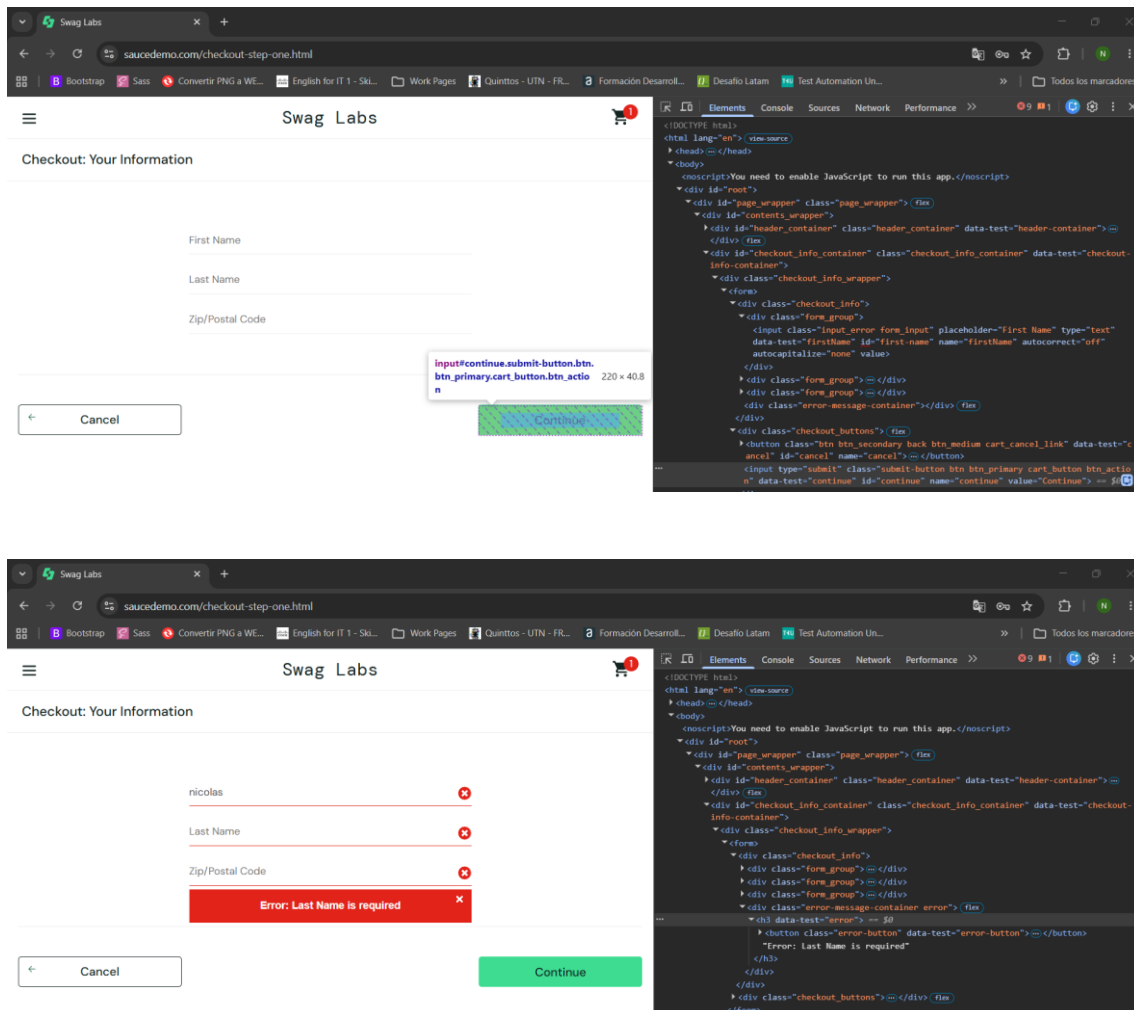
Una vez completada la iteración de agregado masivo de productos, se procedió a simular la navegación hacia el resumen de compra. Mediante la inspección del DOM, se identificó el hipervínculo del carrito de compras. Al interactuar con este elemento, el WebDriver avanza hacia la siguiente pantalla del flujo, estableciendo el estado necesario para validar la correcta persistencia de los ítems seleccionados.



Testing Negativo

Procedí a ejecutar un escenario de prueba negativo en la pasarela de pago (Checkout). El objetivo consistió en validar las restricciones de los campos obligatorios. Mediante la inyección de datos parciales (completando únicamente el campo First Name) y la ejecución de la acción de continuar, se forzó la interrupción del flujo. Posteriormente, se capturó el elemento del DOM correspondiente al mensaje de error (<h3>) para asertar que el sistema devuelve exactamente la cadena de texto esperada ("Error: Last Name is required"), confirmando el correcto manejo de excepciones en la interfaz.





Resolución de validaciones en cascada

Procedí a subsanar el error inicial ingresando los datos requeridos en el campo *Last Name*, con el fin de evaluar el comportamiento del sistema ante múltiples validaciones. Al reenviar el formulario, el script verificó exitosamente que el DOM se actualiza dinámicamente, mutando el mensaje de alerta hacia el siguiente campo vacío. La aserción confirmó la cadena exacta ("Error: Postal Code is required") y la suite de pruebas finalizó de manera exitosa (Status: Passed), validando la robustez de las reglas de negocio del Checkout.

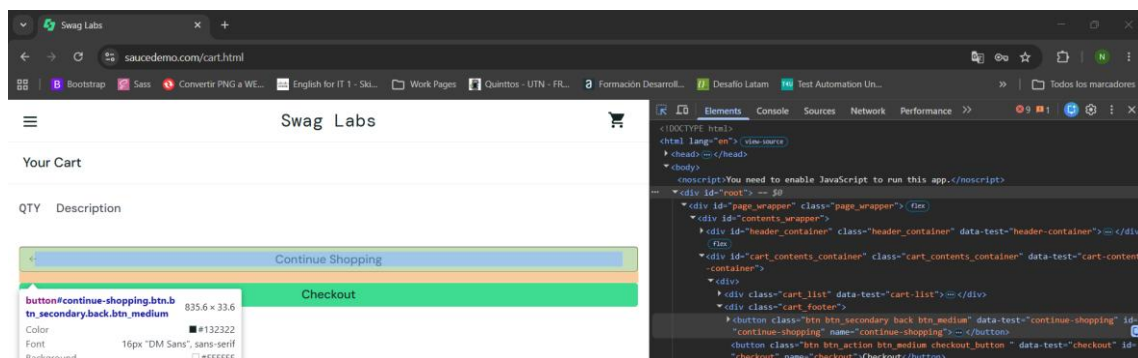
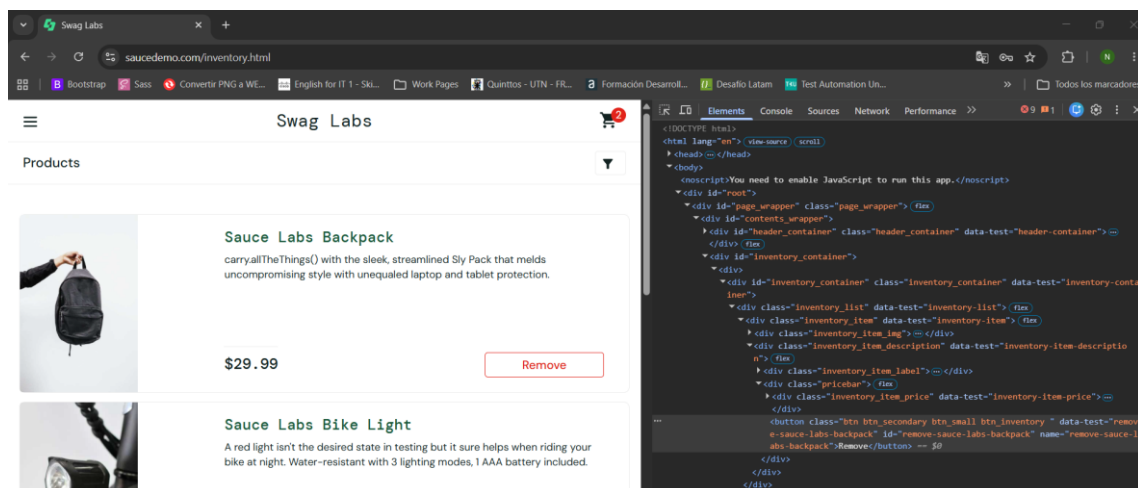
The image shows a Visual Studio Code editor window with the following components:

- EXPLORER:** Displays the file structure of the project `TFI_M3_UNTREF_NICOLAS...`. The files listed are `.pytest_cache`, `test_automation`, `__pycache__`, `conftest.py`, `test_saucedemo...`, `venv`, `punto1.py`, `punto2.py`, `README.md`, and `requirements.txt`.
- EDITOR:** Shows the code for `test_saucedemo.py`. The code defines a function `test_caso2_carrito` that performs several steps:
 - Step 7: Enter name and click "Continue".
 - Step 8: Verify the error "first name".
 - Step 9: Enter last name.
 - Step 10: Verify the error "last name".
- TERMINAL:** Shows the output of running `pytest`. The output indicates that the test session started successfully and that 2 items were collected. The test `test_automation\test_saucedemo.py ..` passed in 53.71s.

TEST CASE 3

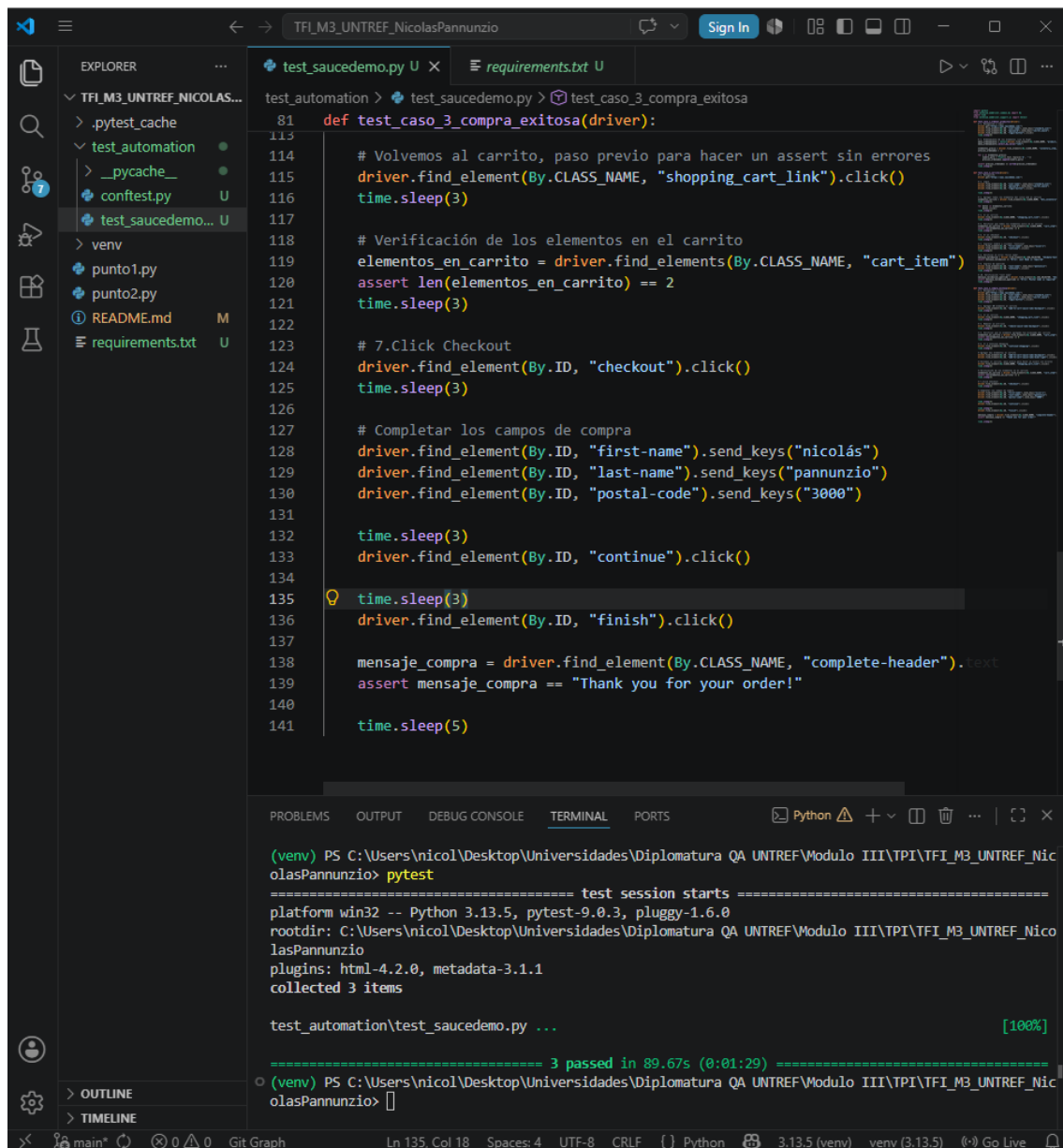
Identificación de controles de eliminación

Para cumplir con el flujo de vaciado del carrito, se inspeccionó el DOM en busca del control de remoción del ítem previamente seleccionado. Se identificó que el botón dinámico posee el atributo ID="remove-sauce-labs-backpack". Al accionar este localizador, se simuló la arrepentimiento de compra, procediendo luego a validar exitosamente mediante aserción matemática ($\text{len}() == 0$) que el nodo del producto fue purgado del DOM del carrito.



Ejecución de flujo End-to-End (E2E) y validación final

Automaticé el recorrido completo de compra del usuario (Happy Path). El WebDriver navegó exitosamente a través del catálogo, gestionó dinámicamente el carrito de compras (altas y bajas de ítems) y completó el formulario de facturación del Checkout. Finalmente, se interactuó con el control de confirmación (`finish`) y se validó mediante aserción la visibilidad del componente `complete-header` con el texto "Thank you for your order!", certificando que la transacción se registró correctamente en el sistema.



The screenshot displays the Visual Studio Code interface. The Explorer panel on the left shows the project structure with files like `test_automation`, `conftest.py`, and `test_saucedemo.py`. The main editor window shows the content of `test_saucedemo.py`, which includes a test function `test_caso_3_compra_exitosa` that performs an end-to-end test. The test steps include clicking the shopping cart link, verifying the cart contents, clicking the checkout button, filling out the purchase form (first name, last name, postal code), clicking continue, clicking finish, and finally asserting that the 'complete-header' contains the text 'Thank you for your order!'. The terminal at the bottom shows the execution of `pytest`, indicating that 3 tests passed in 89.67s.

```
def test_caso_3_compra_exitosa(driver):
    # Volvemos al carrito, paso previo para hacer un assert sin errores
    driver.find_element(By.CLASS_NAME, "shopping_cart_link").click()
    time.sleep(3)

    # Verificación de los elementos en el carrito
    elementos_en_carrito = driver.find_elements(By.CLASS_NAME, "cart_item")
    assert len(elementos_en_carrito) == 2
    time.sleep(3)

    # 7.Click Checkout
    driver.find_element(By.ID, "checkout").click()
    time.sleep(3)

    # Completar los campos de compra
    driver.find_element(By.ID, "first-name").send_keys("nicolás")
    driver.find_element(By.ID, "last-name").send_keys("pannunzio")
    driver.find_element(By.ID, "postal-code").send_keys("3000")

    time.sleep(3)
    driver.find_element(By.ID, "continue").click()

    time.sleep(3)
    driver.find_element(By.ID, "finish").click()

    mensaje_compra = driver.find_element(By.CLASS_NAME, "complete-header").text
    assert mensaje_compra == "Thank you for your order!"

    time.sleep(5)
```

```
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TFI_M3_UNTREF_NicolasPannunzio> pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TFI_M3_UNTREF_NicolasPannunzio
plugins: html-4.2.0, metadata-3.1.1
collected 3 items

test_automation\test_saucedemo.py ... [100%]

===== 3 passed in 89.67s (0:01:29) =====
(venv) PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TPI\TFI_M3_UNTREF_NicolasPannunzio>
```

Aislamiento de estado (State Leakage) y limpieza de caché:

Durante la ejecución en cadena de la suite de pruebas, se detectó un comportamiento anómalo derivado de la persistencia de datos en el navegador. La aplicación SauceDemo almacena el estado del carrito de compras en la memoria local, provocando que el "Caso 2" heredara los ítems seleccionados por el "Caso 1", corrompiendo las validaciones.

Para garantizar el principio fundamental de automatización donde "cada test debe ser independiente y determinístico", se implementó una estrategia de aislamiento. Se inyectó un comando directo al navegador a través del WebDriver

(`driver.execute_script("window.localStorage.clear();")`) seguido de un refresco de pantalla (`driver.refresh()`). Esta intervención asegura que cada escenario comience con un entorno completamente limpio, erradicando los falsos positivos por fugas de estado.

Generación de Reporte y Consolidación de Resultados

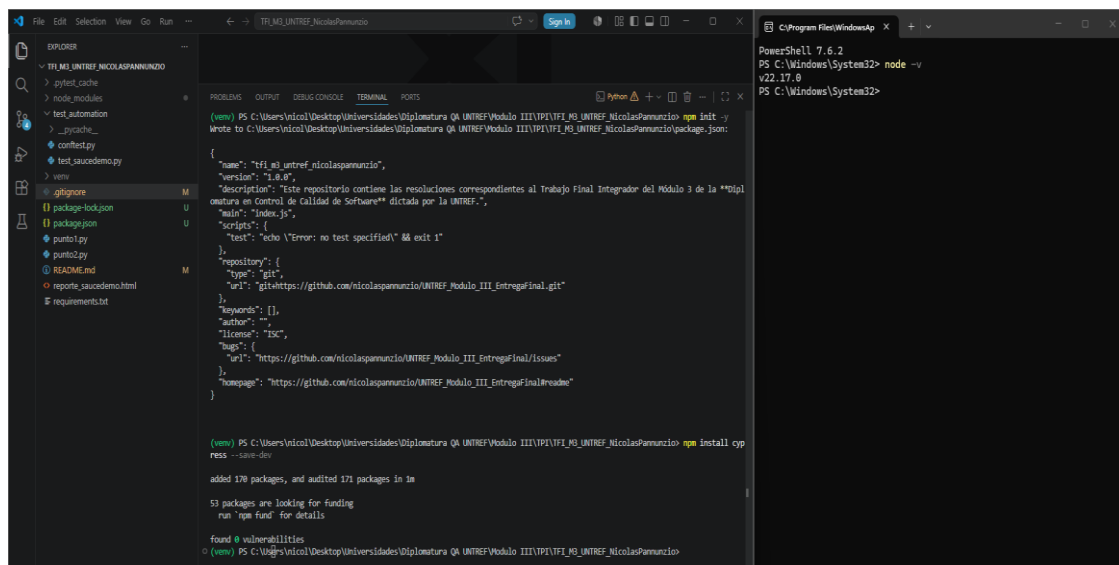
Como cierre de la suite de pruebas End-to-End para el entorno de SauceDemo, se instrumentó la librería `pytest-html` para compilar los resultados de ejecución. Se implementó una estrategia de aislamiento de estado (limpieza de Local Storage) para asegurar la independencia de cada prueba. El reporte final certifica una tasa de éxito del 100% (3/3 Passed) en un tiempo total de ejecución de 1 minuto y 33 segundos, evidenciando un entorno de pruebas estable y listo para integración continua.

reporte_saucedemo.html			
Report generated on 30-May-2026 at 15:33:03 by pytest-html v4.2.0			
Environment			
Python	3.13.5		
Platform	Windows-11-10.0.26200-SP0		
Packages	• pytest 8.0.3 • pluggy 1.6.0		
Plugins	• html 4.2.0 • metadata 3.1.1		
JAVA_HOME	C:\Program Files\Java\jdk-21		
Summary			
3 tests took 00:01:33.			
(Un)check the boxes to filter the results.			
☐ 0 Failed	☒ 3 Passed	☐ 0 Skipped	☐ 0 Expected failures
☐ 0 Unexpected passes	☐ 0 Errors	☐ 0 Reruns	☐ 0 Retries
Result	Test	Duration	Links
Passed	test_automation/test_saucedemo.py::test_caso_1_orderar_productos	00:00:15	
Passed	test_automation/test_saucedemo.py::test_caso_2_carrito	00:00:33	
Passed	test_automation/test_saucedemo.py::test_caso_3_compra_exitosa	00:00:44	

ACTIVIDAD III

CYPRESS – ENTORNO DE PRUEBAS

Para abordar la automatización de las pruebas de backend (PokeAPI), opté por implementar Cypress como framework principal, dada su capacidad nativa para el manejo ágil de peticiones HTTP y evaluación de respuestas JSON. Al coexistir con un entorno previo de Python, se procedió a inicializar el gestor de paquetes de Node.js (npm init) en el directorio raíz, generando el package.json. Posteriormente, se instaló Cypress como dependencia de desarrollo y se protegió el repositorio configurando el archivo .gitignore para excluir el directorio node_modules, garantizando así una arquitectura limpia, aislada y lista para la ejecución de pruebas de API.



```
PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TP1\VF1_M3_UNTREF_NicolasPannunzio> npm init -y
Wrote to C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TP1\VF1_M3_UNTREF_NicolasPannunzio\package.json:

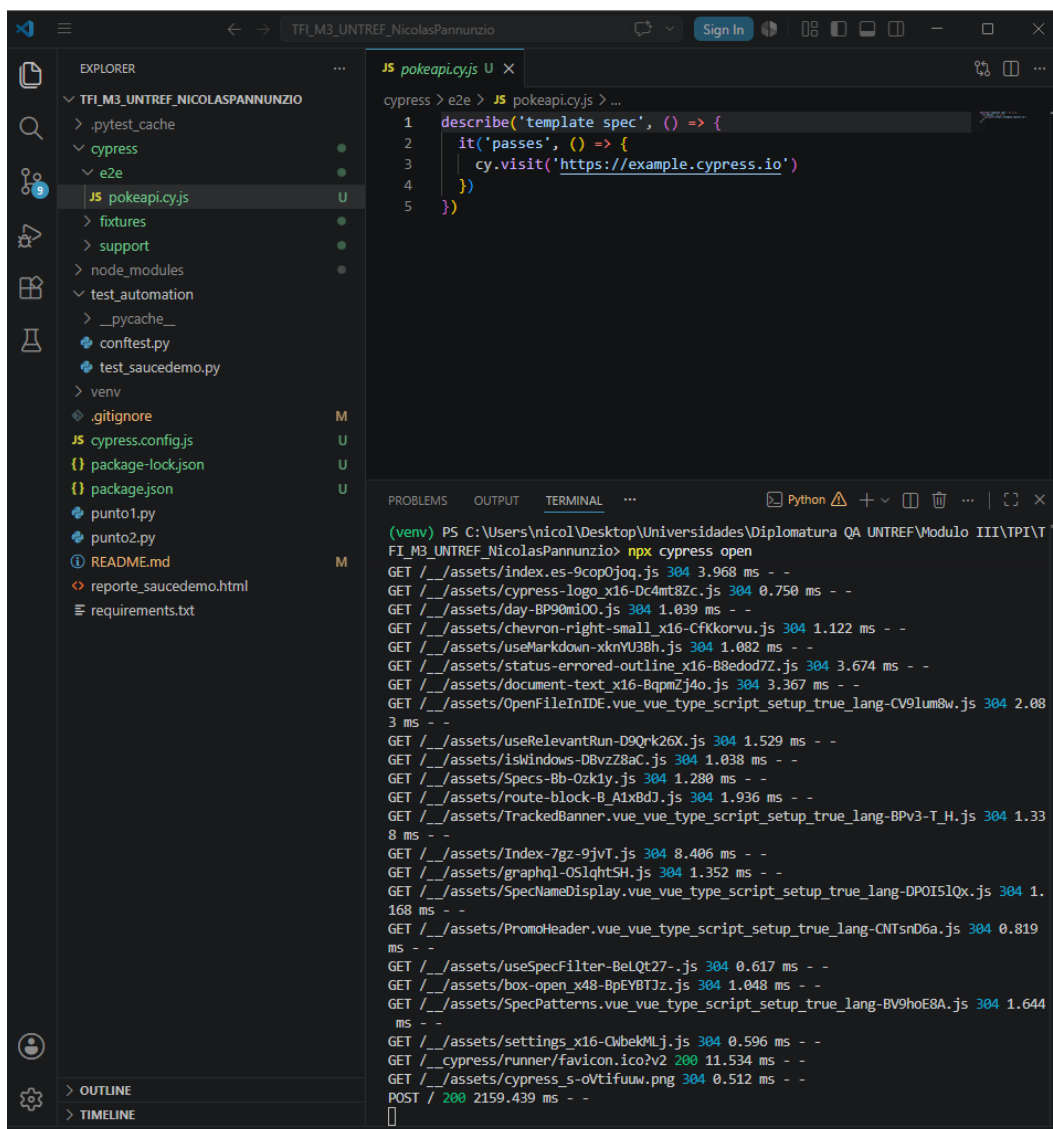
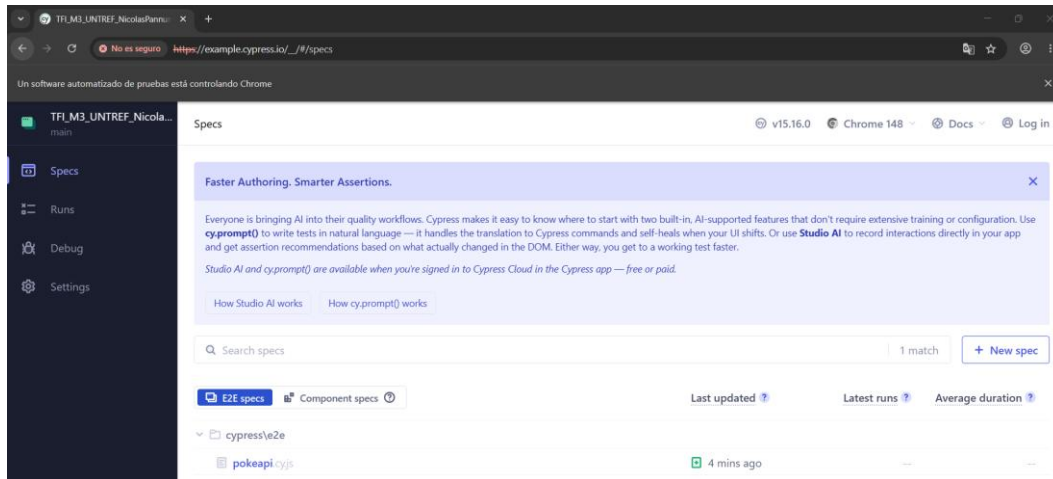
{
  "name": "tp1_m3_untref_nicolaspanunzio",
  "version": "1.0.0",
  "description": "Este repositorio contiene las resoluciones correspondientes al Trabajo Final Integrador del Módulo 3 de la \"Diplomatura en Control de Calidad de Software\" dictada por la UNTREF.",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/nicolaspanunzio/UNTREF_Modulo_III_Entregafinal.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/nicolaspanunzio/UNTREF_Modulo_III_Entregafinal/issues"
  },
  "homepage": "https://github.com/nicolaspanunzio/UNTREF_Modulo_III_Entregafinal#readme"
}

PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TP1\VF1_M3_UNTREF_NicolasPannunzio> npm install cypress --save-dev
added 170 packages, and audited 171 packages in 1m
53 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
PS C:\Users\nicol\Desktop\Universidades\Diplomatura QA UNTREF\Modulo III\TP1\VF1_M3_UNTREF_NicolasPannunzio>
```

Inicialización del test runner y estructura base

Tras la instalación de las dependencias, se ejecutó el entorno gráfico de Cypress mediante el comando de apertura (npx cypress open) en modalidad de pruebas E2E. Este proceso autogeneró la arquitectura de directorios estándar del framework y el archivo de configuración principal (cypress.config.js).

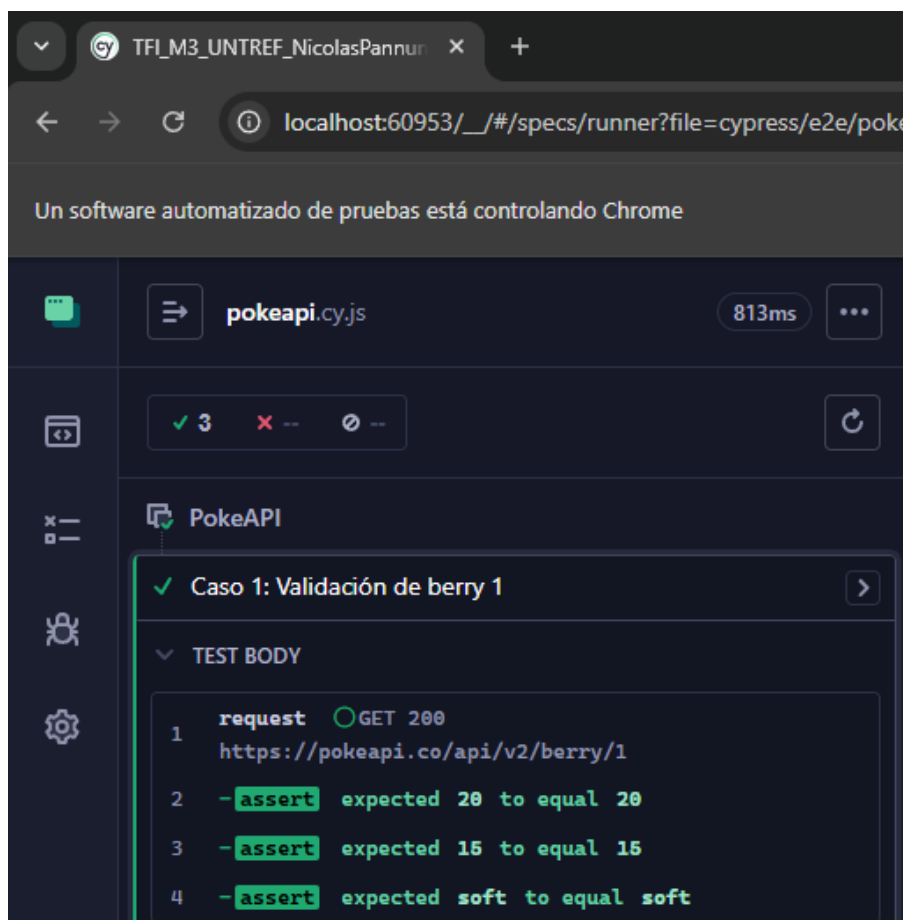
Por último, se instancié el archivo de especificación principal (pokeapi.cy.js) dentro del directorio e2e, dejando el entorno preparado para la redacción de las validaciones de backend.



TEST CASE 1

Ejecución y Validación de API - PokeAPI

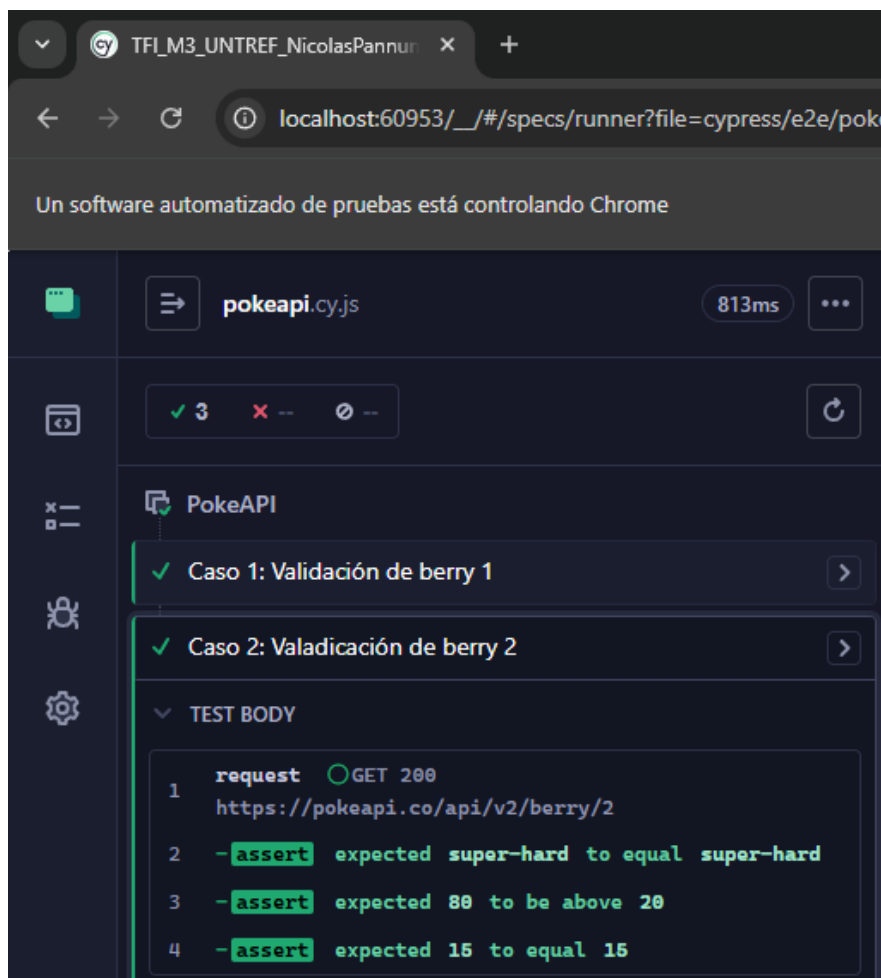
Se desarrolló y ejecutó exitosamente el primer escenario de pruebas de backend sobre la PokeAPI utilizando Cypress. Mediante el comando `cy.request()`, se despachó una petición HTTP GET al endpoint `/berry/1`. Al interceptar la respuesta, se parseó el objeto JSON devuelto y se aplicaron aserciones estrictas utilizando la librería Chai (`expect`). Se comprobó la integridad del payload, confirmando que los atributos `size` (20), `soil_dryness` (15) y el nodo anidado `firmness.name` ("soft") coincidieran con precisión con los criterios de aceptación requeridos. El runner arrojó un resultado exitoso, validando el correcto funcionamiento de este recurso.



TEST CASE 2

Validación de API con Operadores Comparativos

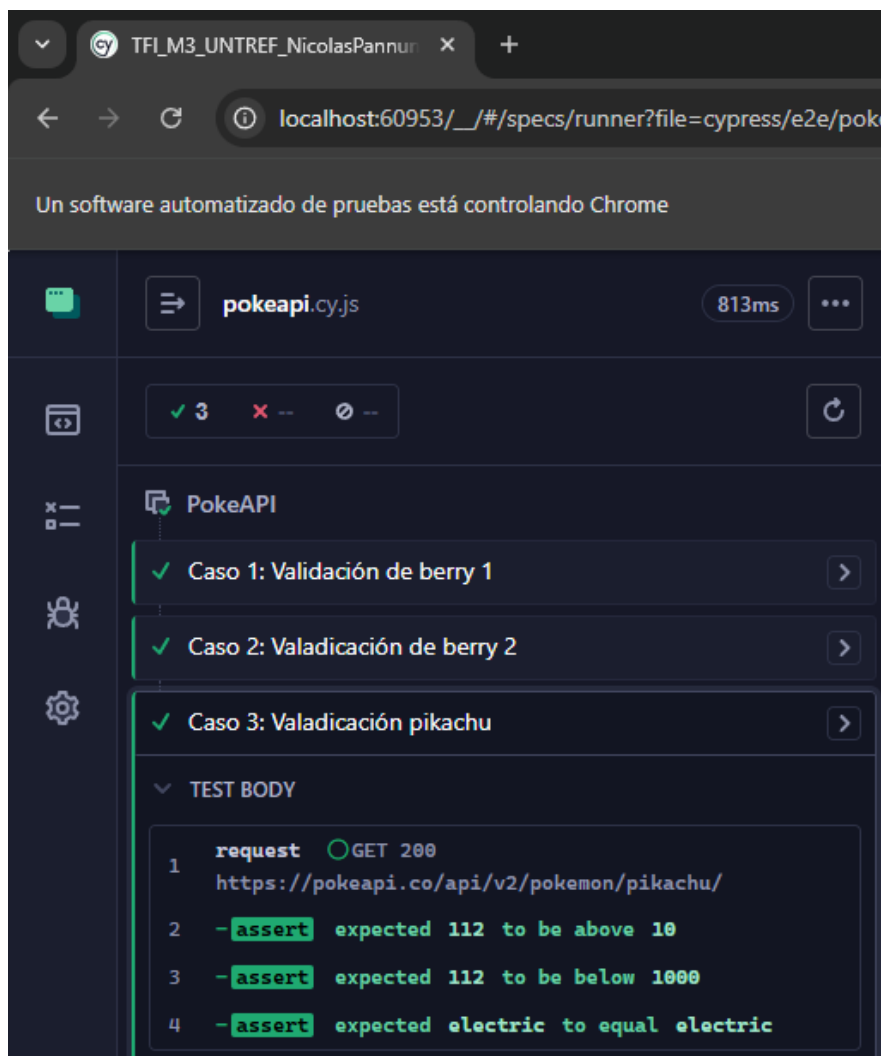
Se desarrolló un segundo escenario de pruebas orientado a la consulta del endpoint `/berry/2`. La validación del payload JSON requirió una combinación de aserciones de igualdad estricta (`.to.eq()`) y operadores matemáticos de comparación (`.to.be.greaterThan()`). Se comprobó exitosamente que el atributo anidado `firmness.name` equivale a "super-hard" y que el parámetro `soil_dryness` mantiene la integridad del caso base (15). Adicionalmente, se verificó de forma dinámica que la propiedad `size` (80) es efectivamente superior al valor registrado en la iteración anterior (`>20`). El test runner finalizó con estado "Passed", validando la lógica comparativa del framework.



TEST CASE 3

Validación estructural

El escenario final de pruebas de backend se ejecutó sobre el endpoint complejo /pokemon/pikachu/. Para validar el payload JSON devuelto, se implementó una estrategia de aserciones encadenadas, confirmando exitosamente que el atributo numérico base_experience (registrado en 112) se ubica dentro del rango dinámico requerido (mayor a 10 y menor a 1000). Finalmente, se parseó la estructura anidada de la respuesta, accediendo al índice inicial del arreglo de características (types[0].type.name) para asertar mediante igualdad estricta que la tipificación del recurso corresponde al valor "electric". La ejecución finalizó con estado "Passed", certificando la robustez de la suite completa.



CONCLUSIONES

Este trabajo integrador permitió aplicar y consolidar conocimientos teóricos y prácticos en lógica de programación, diseño de arquitecturas de pruebas automatizadas y gestión de entornos de desarrollo híbridos. La implementación sobre las plataformas SauceDemo y PokeAPI resultó ser un escenario idóneo para practicar la manipulación dinámica del DOM, el control del estado del navegador y la ejecución de peticiones HTTP para el análisis de respuestas JSON.

Durante el desarrollo del trabajo, se identificaron aspectos clave para garantizar la calidad y robustez del software. Entre ellos destacan: la necesidad de implementar un estricto manejo de excepciones (bloques try-except) en el desarrollo de algoritmos lógicos; la obligatoriedad de mitigar problemas de dependencias de estado mediante la limpieza del Local Storage para asegurar pruebas E2E; y la importancia de parsear y encadenar aserciones precisas sobre estructuras JSON complejas en la validación de APIs REST.

En términos de aprendizaje, el trabajo evidenció el valor de la versatilidad técnica en el rol del QA Automation: desde la capacidad de configurar y aislar entornos virtuales paralelos (Python y Node.js), hasta la instrumentación de librerías para la generación de reportes HTML interactivos. Estas prácticas resultan esenciales para garantizar la calidad, escalabilidad e integración en proyectos de software reales.

HERRAMIENTAS UTILIZADAS

- Sistema operativo: Windows 11 Home.
- Navegador principal: Google Chrome.
- Lenguajes de Programación: Python y JavaScript.
- Frameworks de Automatización: Selenium WebDriver (para pruebas Web UI End-to-End) y Cypress (para pruebas de integración backend/API).
- Gestores de Pruebas y Reportes: Pytest y el plugin pytest-html para la generación de métricas de ejecución.
- Entornos y Gestores de Dependencias: Entornos Virtuales de Python (venv) y gestor de paquetes de Node.js (npm).
- Control de Versiones: Git y GitHub para la gestión del repositorio.
- Editor de Código: Visual Studio Code.
- Documentación: Microsoft Word (elaboración del informe) y Adobe PDF (exportación y entrega formal).
- Uso de IA: Se empleó asistencia de generación de texto basada en modelos de lenguaje para revisar la redacción y estructurar secciones del informe (edición y sugerencias), manteniendo el control humano absoluto sobre el código fuente, la lógica técnica y los resultados experimentales.

BIBLIOGRAFÍA Y REFERENCIAS

- Python Documentation: The Python Standard Library & Errors and Exceptions.
- Selenium Documentation: WebDriver API, Waits & Locators strategies.
- Cypress Documentation: End-to-End Testing, Assertions & API Requests (cy.request).
- Pytest Documentation: Fixtures configuration & pytest-html plugin documentation.
- PokeAPI Documentation: RESTful API interface and JSON structure guidelines.
- SauceDemo: Entorno de pruebas E2E para interfaces de usuario (UI).
- Material teórico propio del curso: Clases correspondientes al Módulo 3 — Diplomatura en Control de Calidad de Software, UNTREF.